ASSIGNMENT NO:10.5

Name:md inayath

Ht.no:2303A51432 bt.no:21

Task Description -1(Error Detection and Correction)

Task:

Use AI to analyze a Python script and correct all syntax and logical errors.

Sample Input Code:

def calculate_total(nums)

sum = 0 for n in nums

sum += n return total

Expected Output-1:

Corrected and executable Python code with brief explanations of the identified syntax and logic errors.

```python
def calculate_total(nums):
    total = 0  # Changed 'sum' to 'total' to avoid conflict with built-in function
    for n in nums:  # Added a colon at the end of the for loop
        total += n  # Indented this line to be inside the for loop
    return total  # Changed 'total' to 'total' to match the variable name used in the function
```

```
explain explanations The original code had a few issues that needed to be addressed:
1. The variable name 'sum' was used, which is a built-in function in Python

2. The for loop was missing a colon at the end, which is required in Python to indicate the st
3. The line that adds the current number to the total was not indented properly, which would c
4. The return statement was using 'total' instead of 'total', which is the variable name used
```

Task Description -2(Code Style Standardization)

Task:

Use AI to refactor Python code to comply with standard coding style

guidelines. Sample Input Code: def findSum(a,b):return a+b

print(findSum(5,10))

Expected Output-2:

Well-structured, consistently formatted Python code following standard style conventions.

Code screenshot:

```python
def find_sum(a, b):
    return a + b
print(find_sum(5, 10))
```

```
The function findSum takes two parameters a and b, and returns their sum. The print statements call the function with
the arguments 5 and 10, resulting in the output of 15 for each call.
```

Output:

```
15
```

Task Description -3(Code Clarity Improvement)

Task:

Use AI to improve code readability without changing its functionality.

Sample Input Code:

def f(x,y): return

x-y*2

print(f(10,3))

Expected Output-3:

Python code rewritten with meaningful function and variable names, proper indentation, and improved clarity.

Code screenshot:

```python
def calculate_difference(x, y):
    return x - y * 2
result = calculate_difference(10, 3)
print(result)
```

```
print(f(10,3))
explain the error in pointwise manner:
1. The function definition is missing a colon (:) at the end of the line. It should be `def f(
2. The body of the function is not indented. In Python, the code inside a
function must be indented to indicate that it belongs to the function. The line `return x-y*2`
3. The print statement is not indented, which is correct since it is outside the function. How
Here is the corrected code:
def f(x, y):
    return x - y * 2
print(f(10, 3))
When you run the corrected code, it will output `4`, since `10 - 3 * 2` equals `4`.
```

Output:

```
h/3-2/assign1.1.py
4
PS D:\Devons>
```

Task Description -4(Structural Refactoring)

Task: Use AI to refactor repetitive code into reusable functions.

Sample Input Code:

print("Hello Ram")

print("Hello Sita") print("Hello

Ravi")

Expected Output-4:

Modular Python code using reusable functions to eliminate repetition.

Code screenshot:

```python
def greet(name):
    print(f"Hello {name}")
names = ["Ram", "Sita", "Ravi"]
for name in names:
    greet(name)
```

```
1. The original code has repetitive print statements for each name, which is not efficient and
2. The code lacks modularity, making it harder to maintain and update if we want to change the
# Refactored code using a reusable function
def greet(name):
    print(f"Hello {name}")
names = ["Ram", "Sita", "Ravi"]
for name in names:
    greet(name)
# Explanation of the refactored code:
1. The `greet` function takes a name as an argument and prints a greeting message. This elimin
2. The list of names is stored in a variable, making it easy to add or remove names without ch
```

Output:

```
Hello Ram
Hello Sita
Hello Ravi
```

Task Description -5(Efficiency Enhancement)

Task:

Use AI to optimize Python code for better performance.

Sample Input Code:

numbers = [ ] for i in

range(1, 500000):

numbers.append(i * i)

print(len(numbers))

Expected Output-5:

Optimized Python code that achieves the same result with improved performance.

Code screenshot:

```python
# Original Code
numbers = []
for i in range(1, 500000):
    numbers.append(i * i)
print(len(numbers))
# Optimized Code
numbers = [i * i for i in range(1, 500000)]
print(len(numbers))
```

```
numbers = [ ]
for i in range(1, 500000):
numbers.append(i * i)
print(len(numbers))
explain the error in point wise manner
1. The code is not properly indented. In Python, indentation is crucial for defining the scope
2. The code is inefficient because it uses a list to store the squares of numbers from 1 to 49
3. The code can be optimized by using a generator expression instead of a list to calculate th
Optimized Code:
numbers = (i * i for i in range(1, 500000))
print(len(list(numbers)))
In the optimized code, we use a generator expression to calculate the squares of numbers from
```

Output:

```
499999
499999
```